

Object Oriented Programming – CS/CE 224/272

Nadia Nasir



Content

- Why study this course
 - Syllabus
 - Why C++
 - Rule of the game
 - Object oriented vs procedural paradigms
 - Course expectations
 - Hello world!
-

Syllabus

Why C++

- C++ is more performant
 - Particularly popular in graphics and scientific computing
 - In the context of ML, most engines are written in C++ with Python used to make calls to backend engine
 - NOT the language for web or for UI!
 - CS@HU decided C++ was the better choice
 - Object oriented programming is not limited to C++. Python, Java, and other languages also has this concept. But
-

Rules of the game

- Setup your environment as soon as possible
 - Any problems, please ask help immediately
 - Expect the pace of the course to be fast. This has been a challenging course in the past.
 - Just a warning. It will be a significant step up from PFun
 - Don't mean to scare you. Just being honest and setting the expectations
 - If you have any issues, difficulty, or want to discuss anything, please feel free to discuss (via email/MS teams).
 - Please don't expect to get response to your emails instantaneously
 - Please don't expect replies over the weekend
-

Questions?

Introduction

Programmers write instructions in various programming languages to perform their computation tasks such as:

(i) Machine level Language

- Machine code or machine language is a set of instructions executed directly by a computer's central processing unit (CPU).

(ii) Assembly level Language

- An assembly language (or assembler language) is a low-level programming language for a computer, or other programmable device, in which there is a very strong (generally one-to-one) correspondence between the language and the architecture's machine code instructions.

(iii) High level Language

- Such languages are considered high-level because they are closer to human languages and farther from machine languages.
 - High-level language has a higher level of abstraction from the computer, and focuses more on the programming logic rather than the underlying hardware components such as memory addressing and register utilization.
-

Compiled vs interpreted languages

The program written in any programming language must be translated into machine language before it can be executed by the computer.

This is done by special programs called compilers, interpreters, or assemblers that are built into the various programming applications.

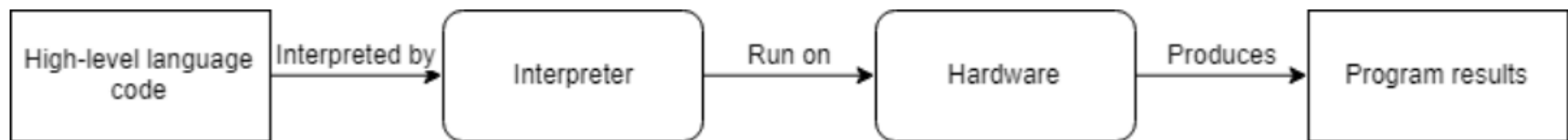
The set of instructions that a programmer writes in any programming language is often called the **source code**.

The source code translation can be done in two ways.

One way is for the computer to translate the source into executable instructions line by line each time the program is run. Language of this type are called **interpreted languages**.

The disadvantage of interpreted languages is that they are slow since the translation is done at the each time the instructions are executed. Python, and MATLAB are examples of interpreted languages.

Here is a simplified representation of the interpretation process

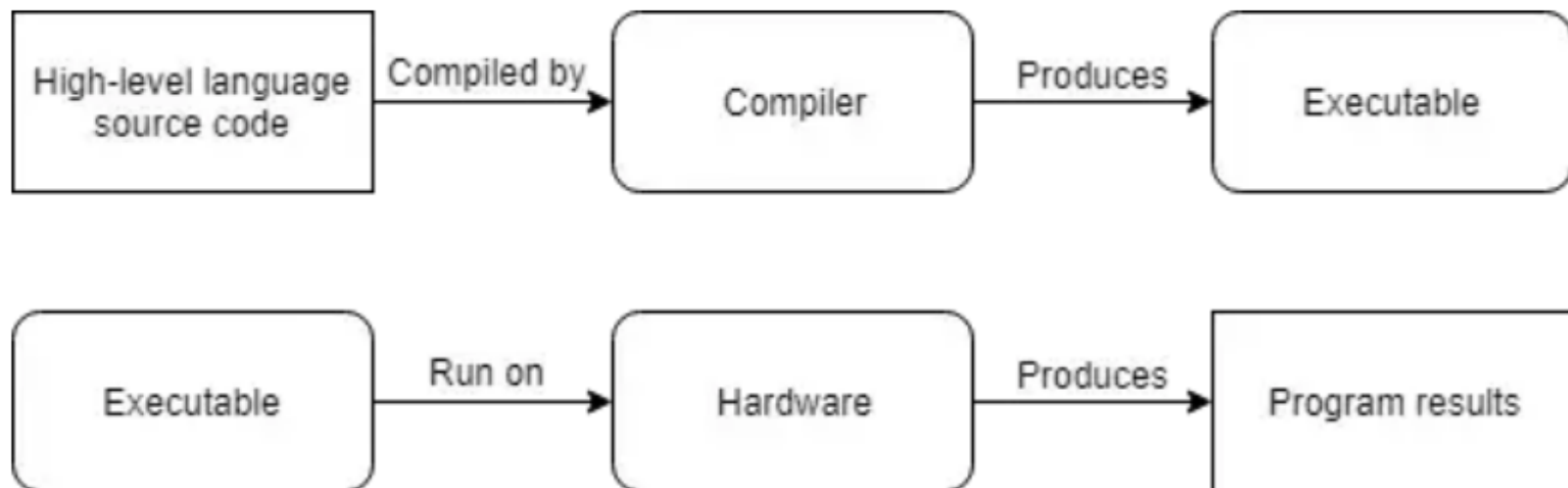


Another way to handle the translation is to compile the source into machine readable programs once and save the machine language instructions in a separate executable file.

When the executable file is run, the translation step is skipped and the program runs much faster than if it were written using an interpreted language.

Programming languages that use this scheme are called **compiled languages**. Examples are C, C++, Java, and Fortran.

Here is a simplified representation of the compiling process:



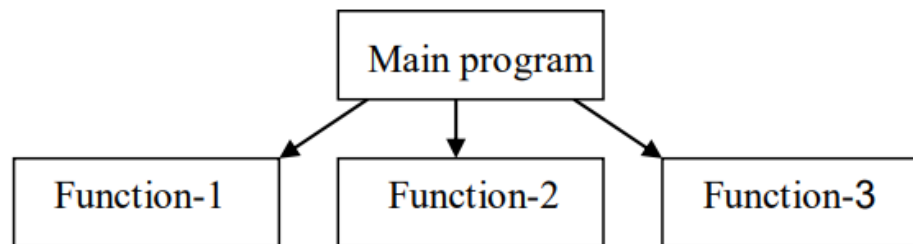
Object oriented vs procedural paradigms

Object oriented vs procedural paradigms



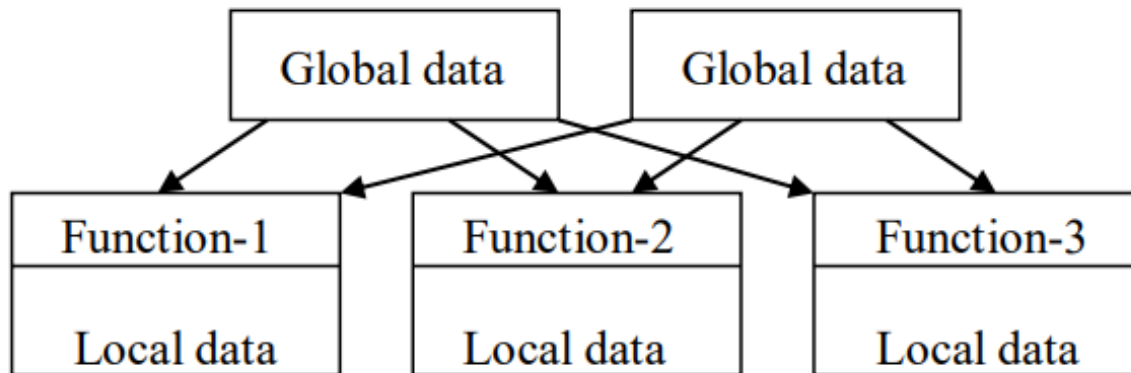
- In procedural paradigms:

- Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and organizing these instruction into groups known as functions.
- Emphasis is on doing things(algorithm).
- Large programs are divided into smaller programs known as functions.
- Most of the functions share global data.
- Data move openly around the system from function to function.
- Function transforms data from one form to another.
- Employs top-down approach in program design.



The disadvantage of the procedure oriented programming languages is:

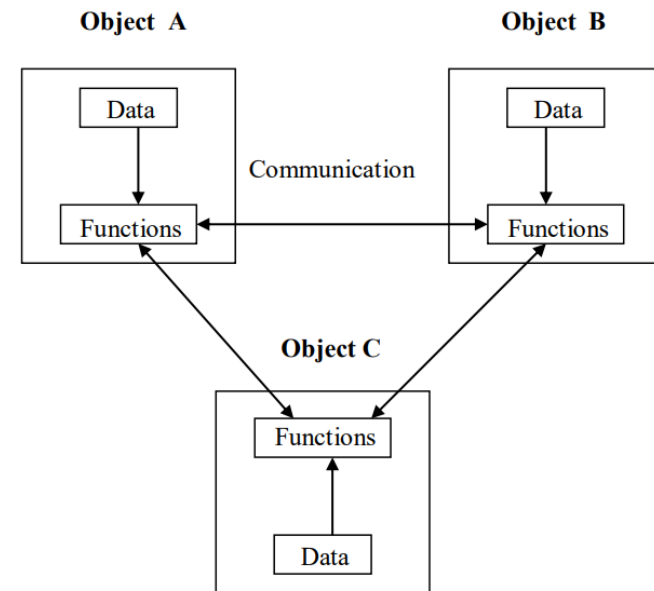
1. Global data access
2. It does not model real word problem very well
3. No data hiding



Object oriented vs procedural paradigms



- In object oriented paradigms:
 - programs are divided into what are known as objects.
 - Data structures are designed such that they characterize the objects.
 - Functions that operate on the data of an object are tied together in the data structure.
 - Data is hidden and can't be accessed by external functions.
 - Objects may communicate with each other through functions.
 - New data and functions can be easily added.
 - Follows bottom-up approach in program design.



Elements of OOP

- Objects
 - Classes
 - Inheritance
 - Reusability
 - ... many other (which we will deal with later)
-

First program in C++

Tools you should install

- On linux, you can install latest version of gcc/g++ - a compiler
- On Windows, I would recommend using MinGW. If you download Visual Studio IDE, then it has its own compiler

Hello C++!

- Your first C++ program

```
#include <iostream>

int main(int argc, char* argv[])
{
    std::cout << "Hello c++!\n";
}
```

- C++ is a compiled language. So you need a compiler!
 - There are many compilers. We use g++
- What can say about this code?

Hello C++!

- Your first C++ program

```
#include <iostream>

int main(int argc, char* argv[])
{
    std::cout << "Hello c++!\n";
}
```

- It has a include statement -> similar to imports in Python
- A C++ must always have a 'main' function. This is the entry point of the code
- 'main' takes 2 arguments.
 - 'argc' counts how many arguments are given to the main function
 - 'argv' what arguments are specifically provided

Hello C++!

- Your first C++ program

```
#include <iostream>

int main(int argc, char* argv[])
{
    std::cout << "Hello c++!\n";
}
```

- cout is used to print to the console.
- cout is an object and hence it has other member functions.
 - More on this later. For now, you can remember how to print to console
 - It will be useful for debugging!
- How does C++ know where to find cout?
 - Hence the include statement. We are telling the compiler that cout is present in iostream, go find there
- Curly braces denote a code block. Python uses indentation!

Next Lecture

- Loops
 - If/else
 - switch
 - Logical and arithmetic operators
 - Datatypes
 - int, size_t, double, float, bool, and char
 - Cast and type conversion
 - Definition and declaration
 - Setw
 - math functions
-